FoodOS: Complete Project Documentation — From Basics to Advanced

A beginner-friendly guide to the actual FoodOS source, Java API, React clients, and MySQL schema

Prepared from the project files inspected on **16 October 2026**. This guide describes the checked-in code and SQL, not an idealized redesign. Where a feature is only partially connected or was not tested, that limitation is stated.

> **Keep credentials private.** The project contains a local `config.properties` file. This guide does not read, copy, or reproduce its values. Never commit that file, passwords, session tokens, or password hashes.

Contents

1. Introduction to FoodOS and its features
2. Basic architecture: frontend, backend, and database
3. Project structure and important files
4. Technologies and dependencies
5. Frontend architecture and API integration
6. Java fundamentals used in this project
7. Object-oriented programming mapped to FoodOS
8. Java backend layers and responsibilities
9. REST APIs, JSON, and HTTP status codes
10. JDBC, SQL statements, and transactions
11. MySQL schema and relationships
12. SQL operations in FoodOS
13. End-to-end request flows
14. Installation and Windows PowerShell run commands
15. Testing, troubleshooting, and incomplete work
16. Security and data-handling basics
17. Glossary, revision notes, and viva questions

How to read this guide

Start with Chapters 1–5 to understand what runs in the browser and what runs in Java. Chapters 6–10 explain the Java and HTTP/JDBC concepts. Chapters 11–13 follow the actual database and request paths. Chapters 14–17 help you run, test, troubleshoot, and explain the project.

“Implemented” means the code path exists in the source. It does **not** automatically mean that a complete integration test was run against MySQL. This guide separates source inspection, unit-test evidence, and live API evidence.

1. Introduction to FoodOS and its features

FoodOS is a food-ordering application. A customer can browse restaurants and menu items, maintain a cart, submit an order, and review order history. Administrative screens manage restaurants, menus, and order status. The repository also contains a Java Swing desktop application, a Java HTTP API, MySQL scripts, and **two different React frontends**.

Main pieces

Piece	What it does
Intended web client	React 19 + TypeScript + TanStack Start/Router app in `frontend/foodos-frontend/`; includes customer and admin routes.
Separate root web client	React 18 + React Router app in `src/`; a distinct implementation with some browser-local data paths.
Java HTTP API	JDK `HttpServer` mounted at `/api`; exposes authentication, restaurant, menu, cart, order, and admin routes.
Java desktop client	Swing application launched by `com.foodapp.Main`; it is separate from the browser apps.
MySQL	Stores users, restaurants, menus, inventory, customer addresses, carts, orders, payments, reviews, history, and audit records.

Features present in source

- Customer registration and password-based login.
- Role-aware API checks for customer and administrator operations.
- Restaurant and menu browsing, with availability and restaurant status.
- Cart operations in both the backend API and browser UI; see Chapter 5 for the important storage distinction.
- Cash-on-delivery order placement, server-side price lookup, inventory checks, transactional order writes, and customer order history.
- Admin endpoints for restaurant/menu CRUD, order status changes, and an admin dashboard summary.
- Java Swing login, registration, browsing, cart, checkout, staff, and administration-oriented panels.

Features not to overstate

- The intended nested web checkout currently offers **Cash on Delivery** only. The schema has simulated payment enum values, but there is no live card or UPI processor.
- The browser carts are stored in browser state/local storage and submitted as order items. Although customer cart endpoints exist, the inspected web cart flows do not use them.
- The root React client and the nested intended client are separate apps; some root-admin modules use IndexedDB/local seed data rather than the Java API.
- Mock data is a preview path, not MySQL. In the nested app it is opt-in via `VITE_USE_MOCK=true`; the provided `.env.example` keeps it disabled.
- A review table and other SQL routines exist, but the existence of a table or routine alone does not prove that the web/API workflow uses it.

2. Basic architecture: frontend, backend, and database

Think of FoodOS as three main layers:

1. **Frontend:** renders pages, collects user input, and displays results.
2. **Backend:** validates requests, applies business rules, checks identity and role, and decides what data may be read or changed.
3. **Database:** stores durable records and enforces relationships and constraints.

An **API** is the contract between frontend and backend. It lets the browser request data over HTTP rather than connecting to MySQL directly.

```
Customer or administrator
          |
          v
React page -> typed frontend service -> HTTP / JSON
                                        |
                                        v
                          Java JDK HttpServer (/api)
                             |              |
                             v              v
                        Services       API routing,
                             |          validation and
                             v          authorization
                           DAO / JDBC
                             |
                             v
                           MySQL

Separate path: Java Swing UI -> Java services / JDBC -> MySQL
```

The intended local browser flow is:

```
Browser at Vite dev server
  -- /api request --> Vite proxy at http://localhost:8080
  -- HTTP request --> FoodOS Java API
  -- JDBC ---------> MySQL database food_ordering
```

The browser must not contain database credentials. MySQL is accessed by Java, not by React.

3. Project structure and important files

The following tree shows the application-level structure. It omits dependency folders such as `node_modules` and generated build output.

```
project/
|-- pom.xml                          Java/Maven build
|-- config.properties               local backend settings; keep private
|-- index.html                      root React app's browser shell
|-- package.json                    separate React 18 frontend
|-- vite.config.ts                  root frontend and /api dev proxy
|-- src/
|   |-- main.tsx, App.tsx           root React entry and router
|   |-- hooks/, lib/, pages/        root client UI and local/API helpers
|   `-- test/, types/, data/        root client tests, types, seed data
|-- frontend/
|   `-- foodos-frontend/
|       |-- package.json            intended React 19 client scripts
|       |-- vite.config.ts          TanStack setup and Java API proxy
|       |-- .env.example            safe local frontend defaults
|       `-- src/
|           |-- routes/             customer and admin file routes
|           |-- services/           API transport and domain calls
|           |-- context/            browser auth/cart providers
|           |-- lib/                cart calculations and helpers
|           |-- mock/               optional browser demo adapter
|           `-- types/              client-side API/data types
|-- src/main/java/com/foodapp/
|   |-- api/, config/, dao/, model/
|   |-- service/, util/, view/
|   `-- Main.java                   Swing desktop entry point
|-- src/test/java/                  Java unit tests
|-- database/                       schema, routines, seed and migration SQL
`-- docs/                           architecture, setup, and this guide
```

Backend entry points and core Java files

Relative path	Purpose and connections
`src/main/java/com/foodapp/Main.java`	Java desktop `main` method. Loads runtime configuration, creates authentication dependencies, and opens the Swing login window. The shaded JAR manifest points here, so `java -jar ...` starts the desktop app rather than the web API.
`src/main/java/com/foodapp/api/ApiMain.java`	Web API `main` method. Builds `Database`, DAO/service objects, `SessionStore`, and `ApiHandler`, then starts the JDK HTTP server on the configured port.
`src/main/java/com/foodapp/api/ApiHandler.java`	HTTP boundary: dispatches routes, reads JSON, validates request values, calls business services, enforces endpoint roles, maps rows/objects to JSON, and writes HTTP status/envelope responses. Some administrator SQL is implemented here directly rather than through a DAO.
`src/main/java/com/foodapp/api/SessionStore.java`	Keeps issued API bearer-token sessions in a process-local concurrent store, looks them up, and revokes them on logout. Sessions disappear when the server restarts.
`src/main/java/com/foodapp/config/AppConfig.java`	Loads `config.properties` from the Java process working directory and falls back to the named environment variables. It supplies DB URL/user/password and API port without putting those values into frontend code.
`src/main/java/com/foodapp/util/Database.java`	Owns the JDBC connection-opening responsibility and loads the MySQL JDBC driver. Its `openConnection()` uses configured values with `DriverManager`.
`src/main/java/com/foodapp/service/AuthenticationService.java`	Implements registration and login business logic. Coordinates user persistence and BCrypt password hashing/verification; used by both the API and desktop login/register screens.
`src/main/java/com/foodapp/service/RestaurantService.java`	Restaurant search/read logic over a DAO; used by API and desktop browsing code.

Relative path	Purpose and connections
`src/main/java/com/foodapp/service/MenuItemService.java`	Menu lookup and menu-item business operations used by restaurant browsing and API routes.
`src/main/java/com/foodapp/service/CartService.java`	Backend cart reads and mutations; checks quantities, availability, and the single-restaurant cart rule. Browser cart state is a separate path.
`src/main/java/com/foodapp/service/OrderService.java`	Validates checkout and coordinates transaction-bound order, order-item, payment, and inventory operations. It also provides order-history/detail operations.
`src/main/java/com/foodapp/service/DashboardService.java`	Builds dashboard summaries from database query results for the desktop application.
`src/main/java/com/foodapp/dao/UserDao.java`, `RestaurantDao.java`, `MenuItemDao.java`, `CartDao.java`, `OrderDao.java`, `AddressDao.java`	Interfaces defining data-access operations independently of the JDBC implementation. Services depend on these contracts.
`src/main/java/com/foodapp/dao/JdbcUserDao.java`, `JdbcRestaurantDao.java`, `JdbcMenuItemDao.java`, `JdbcCartDao.java`, `JdbcOrderDao.java`, `JdbcAddressDao.java`	JDBC implementations. They use SQL and prepared statements to read/write the corresponding MySQL entities. `JdbcOrderDao` receives the service's connection so writes can share a transaction.
`src/main/java/com/foodapp/model/User.java`, `Role.java`	Authenticated user data and the role enum used by authorization logic. The password hash is not part of the client-facing user model.
`src/main/java/com/foodapp/model/Restaurant.java`, `MenuItem.java`	Typed restaurant/menu records passed between DAOs, services, desktop UI, and API mapping.
`src/main/java/com/foodapp/model/Cart.java`, `CartItem.java`	Cart representation and cart lines; `Cart` provides subtotal and quantity calculations used by logic/tests.
`src/main/java/com/foodapp/model/OrderSummary.java`, `OrderDetail.java`, `OrderStatus.java`, `PaymentMethod.java`	Typed order summary/detail data and enums for order/payment states.
`src/main/java/com/foodapp/model/DashboardSummary.java`, `AdminDashboardSummary.java`	Values for dashboard cards and aggregate reporting.
`src/main/java/com/foodapp/util/SessionContext.java`	Desktop-only current-user holder. It is distinct from the HTTP API's bearer-token `SessionStore`.

Desktop UI files

The Swing views under `src/main/java/com/foodapp/view/` are a second Java client, not API routes.

File	Responsibility and concepts
`LoginFrame.java`, `RegisterDialog.java`	Login/registration forms call authentication services and report validation/errors.
`AppFrame.java`, `Sidebar.java`, `UiTheme.java`, `Components.java`	Compose the desktop window, navigation, reusable Swing widgets, and visual styling.
`CustomerDashboardPanel.java`, `StaffDashboardPanel.java`, `AnalyticsPanel.java`	Display role-specific summaries and aggregate values.
`RestaurantBrowsePanel.java`, `MenuManagementPanel.java`, `InventoryPanel.java`	Display restaurant/menu/inventory data and expose management actions.
`CartPanel.java`, `CheckoutDialog.java`	Gather cart and delivery input and initiate desktop checkout.
`OrdersPanel.java`, `OrderDetailDialog.java`, `NotificationsPanel.java`, `SettingsPanel.java`	Display order information, notifications, and account/profile settings.

Many desktop DB tasks use `SwingWorker`, keeping blocking database work away from Swing's event-dispatch thread. Some desktop management/reporting views issue SQL directly; not every operation passes through the service/DAO boundary.

Important frontend files

Relative path	Purpose and connections
`index.html`	HTML shell for the root React 18 client; loads `/src/main.tsx` into the root element. The tagged file is this browser entry shell, not the nested Vite app. Its social-preview image metadata still points to a generic Bolt asset; replace that image before publicly sharing the root client.
`src/main.tsx`, `src/App.tsx`	Root client bootstrapping and `HashRouter` route definitions.
`src/lib/api.ts`	Root client fetch wrapper, session helpers, endpoint calls, and API-to-UI mapping.
`src/hooks/useCart.ts`	Root client cart persisted in browser `localStorage`; computes client-side totals and submits order items.
`src/lib/db.ts`, `src/data/seed.ts`	Root frontend's local IndexedDB/seed-data path, used by some root screens. The seed module is imported; it is not an unused file.
`frontend/foodos-frontend/src/router.tsx`, `src/routeTree.gen.ts`	TanStack router setup and its generated file-route tree. `routeTree.gen.ts` is generated; do not hand-edit it.
`frontend/foodos-frontend/src/routes/__root.tsx`	Shared nested-app shell, providers, and outlet for all file-based routes.
`frontend/foodos-frontend/src/services/api.ts`	Shared HTTP request helper, API envelope handling, token helpers, error type, and opt-in mock-mode selection.
`frontend/foodos-frontend/src/services/authService.ts`	Typed registration/login/logout calls and auth response mapping.
`frontend/foodos-frontend/src/services/restaurantService.ts`, `menuService.ts`, `orderService.ts`	Typed restaurant, menu, checkout, history, and admin service operations.
`frontend/foodos-frontend/src/context/AppProviders.tsx`	Shared auth/cart context; keeps display user/cart in browser storage, enforces one restaurant per cart, and clamps quantity.
`frontend/foodos-frontend/src/lib/cart.ts`	Pure cart arithmetic (`clampQty`, subtotal, estimated total); separately unit-tested.
`frontend/foodos-frontend/src/routes/checkout.tsx`, `orders.tsx`	Checkout form submits an item list and only reports success after the API result; orders page reads the authenticated history.
`frontend/foodos-frontend/src/routes/admin.restaurants.tsx`, `admin.menu.tsx`, `admin.orders.tsx`	Admin forms call typed services for restaurant/menu/order operations. UI role gating is not a substitute for backend authorization.
`frontend/foodos-frontend/src/mock/mockAdapter.ts`	Delayed, local preview data used only if mock mode is explicitly enabled. It is not connected to MySQL.
`frontend/foodos-frontend/src/types/index.ts`	Client-side TypeScript interfaces for users, restaurants, menus, orders, and request/response data.

Relative path	Purpose and connections
`frontend/foodos-frontend/src/lib/cart.test.ts`, `src/test/app-routing.test.tsx`	Cart arithmetic and minimal routing tests; neither exercises the live API or database.

SQL and documentation files

Relative path	Purpose
`database/schema.sql`	Creates `food_ordering` and the 15 relational tables. Primary source for table/column/constraint claims in Chapter 11.
`database/indexes.sql`	Adds secondary indexes. It uses ordinary `CREATE INDEX`, so rerunning after creation may fail on duplicate index names.
`database/functions.sql`	Defines SQL fee/discount/total helper functions. The Java checkout path calculates the current restaurant fee in Java; source search did not show Java invoking these SQL functions.
`database/triggers.sql`	Defines triggers that append order-status history on order creation/status change.
`database/procedures.sql`	Defines order-history and order-status procedures. The API/Java status path performs its own validation; the status procedure itself is not a replacement for that authorization.
`database/views.sql`	Defines customer/order, restaurant revenue, popular-item, and customer-spending views. Their presence does not show that the frontend/API queries them.
`database/seed.sql`, `database/sample_catalog.sql`	Add or upsert demonstration users/catalog/inventory. Do not rerun on a database with valuable data without reading the statements: some inventory seed behavior can restore quantities.
`database/day2_cart_migration.sql`	Migration for an older schema. A new `schema.sql` already defines cart tables, so do not apply migrations blindly to a fresh/current database.
`docs/ARCHITECTURE.md`, `docs/ER_DIAGRAM.md`, `docs/DBMS_CONCEPTS.md`, `docs/JAVA_CONCEPTS.md`	Supporting architecture and teaching notes. A few older statements overstate live verification or cart persistence; use the actual source/schema described here.
`docs/DEPLOYMENT.md`, `docs/PROJECT_STRUCTURE.md`	Deployment and directory guidance. Local configuration values are intentionally not copied into documentation.

4. Technologies and dependencies

Technology	Where used	Why it is present
Java 17	Backend and desktop source	Main language for API, business rules, JDBC, and Swing UI.
Maven	`pom.xml`	Compiles Java, resolves declared dependencies, runs JUnit tests, and packages a shaded JAR.
JDK `HttpServer`	`ApiMain`, `ApiHandler`	Built-in HTTP server; the backend does not use Spring Boot or another web framework.
JDBC + MySQL Connector/J 9.0.0	Java persistence	Sends parameterized SQL to MySQL and reads result sets.
jBCrypt 0.4	Authentication service	Hashes and verifies passwords using BCrypt.
Gson 2.11.0	API JSON boundary	Converts HTTP JSON bodies and Java JSON values.
JUnit Jupiter 5.10.2	Java tests	Unit-testing framework.

Technology	Where used	Why it is present
React 18 + React Router 6	Project-root frontend	Separate older web client.
React 19 + TanStack Start/Router/Query	`frontend/foodos-frontend/`	Intended client routes, server/start integration, and asynchronous query caching.
TypeScript	Both frontends	Adds static types to components, services, and API data.
Vite	Both frontends	Local development server and production asset bundling. Nested frontend is Vite 8; root is Vite 5.
Tailwind CSS, Radix UI, Lucide React	Nested frontend	Styling utilities, accessible UI primitives, and icons.
React Hook Form + Zod	Nested frontend	Form state and schema validation where used.
Vitest, Testing Library, jsdom	Nested frontend tests	Component and utility test infrastructure.
MySQL 8+ / InnoDB	Database	Relational storage, foreign keys, checks, transactions, generated columns, and locking.

The dependency manifests are the authority for exact versions/ranges: root `pom.xml`, root `package.json`, and `frontend/foodos-frontend/package.json`. Lockfiles should be used for reproducible frontend installs.

5. Frontend architecture and API integration

The two clients

These are not two build targets for the same source. The root app uses React 18 and `HashRouter`, while the nested app uses React 19 and TanStack file routes. They have different service modules and local-storage conventions.

Root React 18 routes	Intended nested TanStack routes
`#/`, `#/restaurant/:id`, `#/cart`, `#/orders`, `#/settings`, `#/admin/*`	`/`, `/restaurants`, `/restaurants/$id`, `/menu`, `/cart`, `/checkout`, `/orders`, `/account`, `/login`, `/register`, `/admin` and admin subroutes

Nested app's request path

1. A route/component invokes a typed service function.
2. `services/api.ts` chooses real `fetch` by default, adds request headers and the stored session token when required.
3. The development Vite server proxies `/api` to `http://localhost:8080`.
4. The API handler checks request method/path and role, calls Java logic, and responds in the `{success,data}` envelope.
5. The service returns typed data to React Query/component state.

The default API base is `/api`. `VITE_USE_MOCK=false` keeps mock mode disabled. A real API failure is an error; it must not silently be represented as a successful mock response.

Cart storage is not backend cart persistence

The nested `AppProviders` and root `useCart` keep cart state in the browser. Checkout submits `{ items: [{ itemId, quantity }], addressLine, city, postalCode, paymentMethod }` to `POST /api/orders`. The backend re-reads item prices and validates item availability/stock; it does not trust a client total. Backend cart routes (`GET/DELETE /api/cart`, `POST/PUT /api/cart/items`) exist, but the inspected React cart flows do not call them.

Mock-data caveat

Nested mock mode uses browser-local preview data, not the Java API. Source inspection found checkout/admin mock contract gaps: checkout UI sends `itemId` without `restaurantId`, while the mock checkout expects `menuItemId` and `restaurantId`; its success object differs from the API's `{orderId}` response. Some admin menu/category mock reads are also absent. Therefore use real API mode for a credible integration test; do not infer working production behavior from mock mode.

UI authorization vs server authorization

`RequireRole` and hidden/visible controls improve navigation but are not security boundaries. A user can call HTTP routes directly. `ApiHandler` must still verify the bearer session and role for each protected operation.

6. Java fundamentals used in this project

Classes, objects, and methods

A class describes data and operations; an object is one runtime instance. `Restaurant` represents a restaurant record. `RestaurantService` contains restaurant operations. `ApiMain.main()` is a static entry method that creates the API's object graph.

Types and enums

Java types communicate what a value means. `long` represents database identifiers, `String` represents text, and `BigDecimal` is appropriate for monetary values. `Role`, `OrderStatus`, and `PaymentMethod` restrict a value to named choices rather than arbitrary strings.

Interfaces and `Optional`

DAO interfaces describe available persistence operations, while JDBC classes implement them. A method that searches by email can return `Optional<User>` to express "a user may or may not exist" without returning an ambiguous null.

Exceptions and resource management

Database calls can fail with `SQLException`. Java's try-with-resources closes JDBC connections, statements, and result sets even when an exception happens. Transactions must also explicitly commit or roll back.

Collections, generics, and concurrency

Services pass typed lists of restaurants, menu items, and orders rather than untyped object bags. `SessionStore` uses a concurrent map because HTTP requests can be handled on concurrent threads. The Swing UI uses `SwingWorker` to move blocking database tasks off the UI event thread.

Numeric correctness

Prices use SQL `DECIMAL` and Java decimal values rather than binary floating-point totals. The backend recomputes order amounts using trusted database values.

7. Object-oriented programming mapped to FoodOS

OOP concept	Concrete use
Encapsulation	`AppConfig` hides property/environment resolution behind methods such as `databaseUrl()` and `apiPort()`.
Abstraction	`UserDao`, `OrderDao`, and related interfaces describe the persistence operations without exposing SQL to their callers.
Implementation / polymorphism	`JdbcUserDao` and `JdbcOrderDao` implement DAO interfaces; a service can depend on the interface type rather than the concrete SQL class.
Composition	`ApiMain` constructs a handler from services; services are constructed with database and DAO collaborators. The whole feature is assembled from smaller objects.
Inheritance	Swing frames/panels use the Swing class hierarchy, for example a frame or panel specializes the corresponding Swing widget. React components use composition rather than Java-style inheritance.
Enums	Role and lifecycle/payment values are limited to explicit constants and can be matched to SQL `ENUM` values.

The benefit is separation of responsibility: an HTTP route can change without rewriting the SQL model, and a DAO implementation can be substituted without changing the DAO contract. The code is not perfectly layered: administrator CRUD SQL exists inside `ApiHandler`, and some desktop panels talk directly to SQL. The guide calls that out rather than claiming a strict clean-architecture boundary.

8. Java backend layers and responsibilities

```
ApiMain
  -> JDK HttpServer
      -> ApiHandler (path, method, JSON, auth checks, HTTP response)
          -> AuthenticationService / RestaurantService / MenuItemService
             / CartService / OrderService / DashboardService
              -> DAO interface
                  -> Jdbc...Dao
                      -> Database.openConnection()
                          -> MySQL Connector/J
```

The desktop composition is separate:

```
Main -> LoginFrame -> AuthenticationService
                   -> AppFrame -> Swing panels -> services/DAOs -> MySQL
```

API layer

`ApiMain` wires components; it is not the place for request validation. `ApiHandler` is the HTTP adapter: parse a body, verify the route, call a service, then serialize a response. It also contains some admin SQL, which is a layering exception.

Service layer

Services express business operations. `AuthenticationService` governs register/login; `CartService` guards cart rules; `OrderService` coordinates checkout and transaction boundaries. Services should receive trusted IDs from the authenticated session, not take a user identity on trust from arbitrary client JSON.

DAO/repository layer

A DAO (Data Access Object) isolates SQL. `JdbcOrderDao` writes order tables using a connection supplied by the service, which lets all relevant writes participate in one transaction.

Model layer

Models and enums represent structured data shared between persistence, business logic, and UI/API serialization. JSON field names are chosen by the API mapping methods; a Java field name is not automatically the public API contract.

9. REST APIs, JSON, and HTTP status codes

FoodOS uses the JDK HTTP server rather than a framework. The base path is `/api`. Successful non-empty responses use:

```
{
  "success": true,
  "data": {
    "example": "value"
  }
}
```

Errors use `success: false` and a safe `data.error` message. They must not disclose a stack trace, SQL statement, database credential, password hash, or token. A successful `204 No Content` response has no JSON body.

Confirmed route catalogue

Method	Route	Access	Normal response
`GET`	`/api/health`	Public	`200`, status object
`POST`	`/api/auth/register`	Public	`201`, message object
`POST`	`/api/auth/login`	Public	`200`, flattened user fields and issued token
`POST`	`/api/auth/logout`	Bearer token optional; supplied token is revoked	`204`, empty
`GET`	`/api/restaurants`	Public; optional search/cuisine filters	`200`, restaurant array
`GET`	`/api/restaurants/{id}`	Public	`200`, restaurant
`GET`	`/api/restaurants/{id}/menu`	Public; optional search/category filters	`200`, menu array
`GET`	`/api/cart`	Customer	`200`, cart lines
`POST`	`/api/cart/items`	Customer	`204`, empty
`PUT`	`/api/cart/items/{itemId}`	Customer	`204`, empty
`DELETE`	`/api/cart`	Customer	`204`, empty
`POST`	`/api/orders`	Customer; COD is the implemented web/API method	`201`, `{orderId}`
`GET`	`/api/orders/my`	Customer	`200`, own order summaries

Method	Route	Access	Normal response
GET	/api/admin/restaurants	Admin	200, restaurant array
POST	/api/admin/restaurants	Admin	201, {restaurantId}
PUT	/api/admin/restaurants/{id}	Admin	200, message object
DELETE	/api/admin/restaurants/{id}	Admin	204, empty
GET	/api/admin/categories	Admin	200, category array
GET	/api/admin/menu-items	Admin; optional restaurantId	200, menu-item array
POST	/api/admin/menu-items	Admin	201, {itemId}
PUT	/api/admin/menu-items/{id}	Admin	200, message object
DELETE	/api/admin/menu-items/{id}	Admin	204, empty
GET	/api/admin/orders	Admin	200, order array
PUT	/api/admin/orders/{id}	Admin	200, message object
GET	/api/admin/dashboard	Admin	200, summary object

Common HTTP status meanings

Status	Meaning in FoodOS
200 OK	Read or update completed.
201 Created	Registration, order, restaurant, or menu item was created.
204 No Content	Mutation/logout completed with no response body.
400 Bad Request	Missing, malformed, or invalid input; invalid checkout data.
401 Unauthorized	Missing, invalid, expired, or revoked login session.
403 Forbidden	Session is valid but has the wrong role.
404 Not Found	Requested restaurant/menu/order/resource was not found.
409 Conflict	A delete or mutation conflicts with existing data/constraints.
503 Service Unavailable	Backend cannot complete an operation due to a database availability failure.
500 Internal Server Error	Unexpected server failure; details belong in protected logs, not client responses.

The API's status and response envelopes are confirmed from `ApiHandler.java`. The endpoint list describes source; it is not a promise that every UI flow has an automated integration test.

10. JDBC, SQL statements, and transactions

JDBC is Java's standard database interface. Connector/J supplies the MySQL-specific driver. `Database.openConnection()` returns a `Connection`; code creates a `PreparedStatement`, binds values, executes SQL, and reads a `ResultSet`.

```
return DriverManager.getConnection(
    config.databaseUrl(),
    config.databaseUsername(),
    config.databasePassword()
);
```

This is the actual connection-opening pattern in `Database.java`. The credentials come from runtime configuration, not from a frontend request.

Why use `PreparedStatement`?

SQL values are bound as parameters, not concatenated into the SQL string:

```
PreparedStatement statement =
    connection.prepareStatement("SELECT ... WHERE email = ?");
statement.setString(1, email);
```

The `?` is a value placeholder. It helps prevent a user-supplied value from changing the SQL structure. Table/column names are SQL structure and should not be accepted freely from request data.

`ResultSet`

A `ResultSet` is a cursor over rows returned by a query. JDBC DAOs read each row and build a typed model such as `User`, `Restaurant`, or `OrderSummary`. Result sets/statements/connections are closed with try-with-resources.

Transactions

A transaction groups database changes into one all-or-nothing unit:

```
setAutoCommit(false)
  -> lock/read current menu and stock
  -> validate
  -> decrease inventory
  -> insert order
  -> insert order items and payment
  -> commit
on failure -> rollback
```

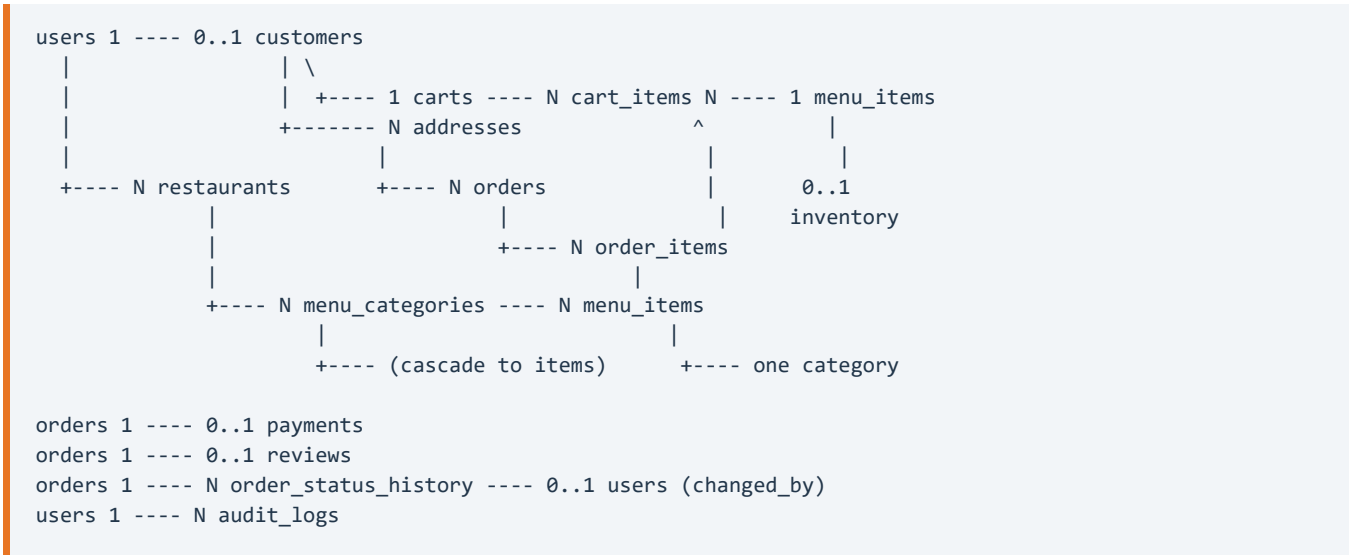
The checkout service uses row locking (`SELECT ... FOR UPDATE`) and guarded inventory changes so concurrent requests cannot oversell the same stock. Item prices are read from the database and order lines store a historical unit-price snapshot. The SQL generated `order_items.line_total` is computed from quantity × unit price. Exceptions roll back the transaction.

The service tests focus on validation before database access. They do not replace a database integration test for transaction rollback, stock contention, or a successfully persisted order.

11. MySQL schema and relationships

`database/schema.sql` creates the `food_ordering` database and **15 InnoDB tables**. `PK` means primary key, `FK` means foreign key, `UQ` means unique constraint, `AI` means auto-increment, and `?` marks a nullable column. Unless listed as nullable, columns are `NOT NULL`.

Entity relationship diagram



The diagram's "0..1" for payment, inventory, and review reflects that the foreign-key row is not mandatory. The schema's unique key allows no more than one payment per order and one review per order.

Tables and columns

Identity and restaurants

1. **users** — `user_id BIGINT PK AI`; `email VARCHAR(150) UQ`; `password_hash VARCHAR(100)`; `full_name VARCHAR(120)`; `role ENUM('CUSTOMER','RESTAURANT_OWNER','RESTAURANT_STAFF','ADMIN') DEFAULT 'CUSTOMER'`; `is_active BOOLEAN DEFAULT TRUE`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.
2. **customers** — `customer_id BIGINT PK/FK -> users.user_id ON DELETE CASCADE`; `phone VARCHAR(25)?`. It extends a user record rather than duplicating email/name/password.
3. **restaurants** — `restaurant_id BIGINT PK AI`; `owner_user_id BIGINT FK -> users.user_id`; `name VARCHAR(150)`; `description TEXT?`; `cuisine VARCHAR(80)`; `location VARCHAR(180)`; `contact_phone VARCHAR(25)?`; `opening_hours VARCHAR(120)?`; `rating DECIMAL(3,2) DEFAULT 0.00`; `delivery_time_minutes INT DEFAULT 30`; `delivery_distance_km DECIMAL(4,1) DEFAULT 0.0`; `delivery_charge DECIMAL(10,2) DEFAULT 0.00`; `status ENUM('PENDING','ACTIVE','SUSPENDED','CLOSED') DEFAULT 'PENDING'`; `cover_image_path VARCHAR(500)?`; `map_address VARCHAR(300)?`; `map_lat DECIMAL(10,7)?`; `map_lng DECIMAL(10,7)?`; `map_link VARCHAR(500)?`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`; `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`. UQ `(owner_user_id,name)`.

Menu and stock

1. **menu_categories** — `category_id BIGINT PK AI`; `restaurant_id BIGINT FK -> restaurants.restaurant_id ON DELETE CASCADE`; `name VARCHAR(80)`; `sort_order INT DEFAULT 0`. UQ `(restaurant_id,name)`.
2. **menu_items** — `item_id BIGINT PK AI`; `category_id BIGINT FK -> menu_categories.category_id`; `name VARCHAR(150)`; `description VARCHAR(500)?`; `price DECIMAL(10,2)`; `discount_price DECIMAL(10,2)?`; `preparation_time_min INT DEFAULT 15`; `is_veg BOOLEAN DEFAULT TRUE`; `availability ENUM('AVAILABLE','SOLD_OUT','HIDDEN') DEFAULT 'AVAILABLE'`; `image_path VARCHAR(500)?`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`; `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`. UQ `(category_id,name)`. Checks: price > 0, discount is null or >= 0, prep time > 0.
3. **inventory** — `item_id BIGINT PK/FK -> menu_items.item_id ON DELETE CASCADE`; `quantity_available INT DEFAULT 0`; `reorder_level INT DEFAULT 5`; `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE`

`CURRENT_TIMESTAMP`. Check quantity >= 0. One row per item at most; an inventory row is not required by the FK.

Customer cart and address

1. **addresses** — `address_id BIGINT PK AI`; `customer_id BIGINT FK -> customers.customer_id ON DELETE CASCADE`; `label VARCHAR(40)`; `address_line VARCHAR(255)`; `city VARCHAR(80)`; `postal_code VARCHAR(15)`; `is_default BOOLEAN DEFAULT FALSE`.
2. **carts** — `cart_id BIGINT PK AI`; `customer_id BIGINT UQ/FK -> customers.customer_id ON DELETE CASCADE`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`; `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`. At most one persisted cart per customer.
3. **cart_items** — `cart_id BIGINT FK -> carts.cart_id ON DELETE CASCADE`; `item_id BIGINT FK -> menu_items.item_id`; `quantity INT`; composite `PK(cart_id,item_id)`. Check quantity > 0. The schema itself does not impose the UI's maximum quantity.

Orders and supporting records

1. **orders** — `order_id BIGINT PK AI`; `customer_id BIGINT FK -> customers.customer_id`; `restaurant_id BIGINT FK -> restaurants.restaurant_id`; `address_id BIGINT FK -> addresses.address_id`; `status ENUM('PLACED','ACCEPTED','PREPARING','READY','OUT_FOR_DELIVERY','DELIVERED','REJECTED','CANCELLED') DEFAULT 'PLACED'`; `subtotal DECIMAL(10,2)`; `discount_amount DECIMAL(10,2) DEFAULT 0.00`; `delivery_fee DECIMAL(10,2) DEFAULT 0.00`; `total_amount DECIMAL(10,2)`; `order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP`. Check the four amount columns are nonnegative.
2. **order_items** — `order_item_id BIGINT PK AI`; `order_id BIGINT FK -> orders.order_id ON DELETE CASCADE`; `item_id BIGINT FK -> menu_items.item_id`; `quantity INT`; `unit_price DECIMAL(10,2)`; `line_total DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) STORED`. Check quantity > 0. `unit_price` snapshots the price charged at order time.
3. **payments** — `payment_id BIGINT PK AI`; `order_id BIGINT UQ/FK -> orders.order_id ON DELETE CASCADE`; `method ENUM('CASH','CARD_SIMULATED','UPI_SIMULATED')`; `amount DECIMAL(10,2)`; `status ENUM('PENDING','SUCCESS','FAILED','REFUNDED') DEFAULT 'PENDING'`; `paid_at TIMESTAMP?`. At most one payment record per order.
4. **reviews** — `review_id BIGINT PK AI`; `order_id BIGINT UQ/FK -> orders.order_id`; `customer_id BIGINT FK -> customers.customer_id`; `restaurant_id BIGINT FK -> restaurants.restaurant_id`; `rating TINYINT`; `comment VARCHAR(500)?`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`. Check rating is 1 through 5. A review is not required.
5. **order_status_history** — `history_id BIGINT PK AI`; `order_id BIGINT FK -> orders.order_id ON DELETE CASCADE`; `old_status VARCHAR(30)?`; `new_status VARCHAR(30)`; `changed_by BIGINT? FK -> users.user_id`; `changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.
6. **audit_logs** — `audit_id BIGINT PK AI`; `user_id BIGINT? FK -> users.user_id`; `action VARCHAR(80)`; `entity_type VARCHAR(50)`; `entity_id BIGINT?`; `details JSON?`; `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.

Unique keys, checks, and deletion behavior

- Unique: user email; restaurant owner/name; category restaurant/name; menu category/name; one cart per customer; one payment per order; one review per order.
- Checks: positive item price/preparation time/cart and order-item quantity; nonnegative discount, stock, and order totals; review rating 1–5.
- Cascades are declared from user to customer; customer to address/cart; restaurant to category; menu item to inventory; cart to cart lines; and order to order items, payment, and status history.

- Other foreign keys use the database's default no-cascade behavior. Deletion can therefore be rejected when historical/related rows still reference a record.
- SQL routines, procedures, triggers, and views are separate schema objects, not tables.

12. SQL operations in FoodOS

Application queries

JDBC DAOs contain parameterized SQL for lookups, inserts, updates, and joins. Typical tasks are:

Task	Data involved	Why it matters
Find an account by email	`users`	Login needs the stored password hash and role; the plaintext password is never queried from the database because it is not stored.
Browse a restaurant menu	`restaurants`, `menu_categories`, `menu_items`, and optionally `inventory`	Joins make category, restaurant, price, availability, and stock displayable together.
Load cart lines	`carts`, `cart_items`, menu/inventory	A persisted cart line references the real item ID and quantity. Browser clients currently keep their active cart separately.
Create an order	`orders`, `order_items`, `payments`, `addresses`, `inventory`	Related writes happen on the same transaction connection.
Read customer order history	`orders` joined with restaurant data and summary fields	The service scopes results to the authenticated customer.
Admin CRUD/status changes	Restaurant/menu/order tables and audit/history	API role checks must happen before the write.

An example of the **shape** of a safe lookup is:

```
SELECT user_id, email, full_name, role, is_active
FROM users
WHERE email = ?;
```

The placeholder represents a bound JDBC parameter. Do not concatenate user input into SQL. In API admin operations, some SQL is inside `ApiHandler`; many other operations live in `Jdbc...Dao` classes.

Additional SQL objects

- `functions.sql` defines `calculate_delivery_fee(subtotal)`, `calculate_discount(subtotal)`, and `calculate_order_total(subtotal)`. Its sample fee function returns zero at the configured subtotal threshold and a fixed lower-subtotal fee. The Java checkout uses each restaurant's configured delivery charge; no Java reference to these SQL functions was found.
- `procedures.sql` defines `get_customer_order_history(customer_id)` and `update_order_status(order_id,new_status,actor)`. The procedure is not a replacement for the API's authorization/transition validation.
- `triggers.sql` appends order-status history when an order is inserted or its status changes.
- `views.sql` defines `CustomerOrderSummary`, `RestaurantRevenueSummary`, `PopularMenuItems`, and `CustomerSpendingSummary`. Source inspection did not show the API using these views.
- `indexes.sql` adds secondary indexes for common searches/joins. It is not safe to assume this file can be run repeatedly because its `CREATE INDEX` statements are not guarded by existence checks.

Idempotency and safe setup

`CREATE TABLE IF NOT EXISTS` does not upgrade an existing table definition. Seed scripts use upserts for several entities, but that does not mean every rerun is a no-op: sample inventory statements can restore stock. Stored functions/procedures/triggers use drop-and-recreate patterns, and views use create-or-replace. Review the exact script and target schema before applying SQL to a database with valuable data. Never use `DROP DATABASE`, `TRUNCATE`, or a seed rerun as a debugging shortcut.

The documented fresh-schema order is:

```
schema.sql
indexes.sql
functions.sql
triggers.sql
procedures.sql
views.sql
seed.sql
sample_catalog.sql
```

The Day-2 cart migration is for an older Day-1 database; the current fresh schema already defines carts. This guide reports script structure from source; it does not claim that every SQL routine was executed in this documentation task.

13. End-to-end request flows

A. Customer registration and login

```
register form
 -> authService.register()
 -> POST /api/auth/register (fullName, email, password, phone)
 -> ApiHandler -> AuthenticationService
 -> BCrypt hash + transactional user/customer/audit writes
 -> MySQL
 <- 201 { success:true, data:{ message } }

login form
 -> authService.login()
 -> POST /api/auth/login (email, password)
 -> AuthenticationService verifies BCrypt and active status
 -> SessionStore issues process-local token
 <- 200 { success:true, data:{ user fields..., token } }
 -> frontend stores token for later protected requests
```

Registration returns a message, not an automatically authenticated user. Login response is flattened in `data`; it is not `{user: ..., token: ...}`. Logout posts to the auth endpoint and revokes the supplied session token.

B. Restaurant and menu browsing

```
restaurants route
 -> restaurantService.list()
 -> GET /api/restaurants
 -> API/RestaurantService/RestaurantDao
 -> SELECT restaurant rows from MySQL
 <- {success:true,data:[...]}

restaurant detail
 -> GET /api/restaurants/{id}
 -> GET /api/restaurants/{id}/menu
 -> menuService maps availability/status fields to UI types
```

The API only returns what the SQL/API mapper selects. The UI must not assume that every optional database column has a corresponding populated value.

C. Cart operations

The browser cart uses React context/local storage and client-side arithmetic. Add/remove/quantity actions update the local cart. The service sends item IDs and quantities during checkout. The backend also exposes database cart endpoints; the inspected frontends do not use those routes for their current cart display.

D. Checkout and order persistence

```
checkout form validates visible fields
 -> POST /api/orders with item IDs, quantities, address, city, postal code, CASH
 -> ApiHandler authenticates customer and validates request shape
 -> OrderService opens a transaction
 -> read/lock current menu and inventory rows
 -> verify available item, restaurant/stock/quantities
 -> compute trusted prices and delivery charge server-side
 -> reduce stock and insert order + order lines + payment
 -> commit (or rollback everything on failure)
 <- 201 {success:true,data:{orderId}}
 -> UI clears cart and shows order history/confirmation
```

The client-supplied total is not authoritative. `unit_price` is recorded on each order line to preserve historical pricing. The frontend should not show a success page unless it receives the server's successful order response.

Repeated submission note: the visible frontend disables the submit action while a request is pending. The inspected API does not expose a documented idempotency-key contract; do not assume that retrying an uncertain request can never create a second order.

E. Customer order history

```
orders page -> GET /api/orders/my + bearer session
 -> backend derives customer identity from session
 -> service/DAO selects only that customer's orders
 <- summaries and status values in the API envelope
```

The customer ID must come from the authenticated server-side session, not from a freely chosen query parameter.

F. Administrator operation

```
admin UI role gate
 -> PUT /api/admin/menu-items/{id} (authenticated request)
 -> ApiHandler resolves token and requires ADMIN
 -> validate/map supplied item fields
 -> parameterized JDBC update
 -> response; UI refetches admin/customer data
```

The same server-side role check applies if someone skips the UI and calls the API directly. Restaurant status, delivery fee, and delivery time are supported by existing schema columns; no schema alteration is needed for those fields.

14. Installation and Windows PowerShell run commands

Prerequisites

- Java 17 or newer.
- Maven 3.8 or newer.
- MySQL 8 or newer, with the MySQL client or Workbench.
- Node.js `^20.19.0` or `>=22.12.0` and npm for the nested Vite 8 frontend.
- A terminal opened at the project root (the directory containing `pom.xml`).

Check installed tools:

```
java -version
mvn -version
node --version
npm --version
```

The MySQL service should be running locally or reachable at the host configured for Java. Create the `food_ordering` schema and a dedicated least-privilege application account using local MySQL administration. Do not put a real password in this guide or in React files.

Load a fresh database

From PowerShell at the project root, open the MySQL command client:

```
mysql -u root -p
```

Enter the administrator password only at the prompt. In the MySQL client, execute the files **in the order in Chapter 12** using `SOURCE` (the MySQL client accepts Windows paths):

```
SOURCE .\database\schema.sql;
SOURCE .\database\indexes.sql;
SOURCE .\database\functions.sql;
SOURCE .\database\triggers.sql;
SOURCE .\database\procedures.sql;
SOURCE .\database\views.sql;
SOURCE .\database\seed.sql;
SOURCE .\database\sample_catalog.sql;
```

Successful statements normally print `Query OK` or an affected-row count. This is for a fresh/dev database only. Before applying to an existing database, inspect current tables/data and the migration path; do not blindly repeat `indexes.sql` or catalog inventory upserts.

Configure the Java API

Java reads `config.properties` from the process's **working directory**. The keys are:

```
db.url
db.username
db.password
api.port
```

Environment fallbacks are `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, and `API_PORT`. A nonblank value in `config.properties` takes precedence over the environment value. Keep the local file private/ignored, use a dedicated DB account, and never paste it into a chat or commit.

Build and start the backend

From the project root:

```
mvn test
mvn package
java -cp .\target\online-food-ordering-system-1.0.0.jar com.foodapp.api.ApiMain
```

- `mvn test` compiles and runs the JUnit tests.
- `mvn package` builds the project and shaded artifact.
- The explicit `ApiMain` class starts the HTTP API. The default shaded-JAR manifest names `com.foodapp.Main`, which starts the Swing desktop UI instead.

In a second PowerShell window, check the API:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/health' -Method Get
```

Expected shape is a successful envelope containing `data.status` equal to `ok`. If `api.port` is changed, update this URL and the Vite proxy target consistently.

Start the intended frontend

Open a second terminal at project root, then:

```
Set-Location .\frontend\foodos-frontend
npm ci
npm run dev
```

`npm ci` installs exactly from `package-lock.json`; `npm run dev` starts Vite and prints the local browser URL. The Vite dev proxy forwards `/api` to `http://localhost:8080`. `.env.example` sets `VITE_API_BASE_URL=/api` and mock mode false.

Run frontend checks from `frontend\foodos-frontend`:

```
npm test
npm run lint
npm run build
```

`npm test` runs Vitest; `npm run lint` runs ESLint; `npm run build` creates the production build. The root frontend is separate; from project root, `npm ci`, `npm run dev`, and `npm run build` apply to that client. The root manifest does not declare a test script.

Production note

The Vite `/api` proxy is for local development; it is not a production reverse proxy. Production needs a deployed Java API, MySQL service, HTTPS, correct frontend API base URL, restricted CORS, and a static host configured for client-side route fallback. The current handler allows broad CORS and should not be exposed unchanged to untrusted production origins.

15. Testing, troubleshooting, and incomplete work

Tests present

Test suite	What it checks	What it does not prove
Java `OrderServiceTest` (3 tests)	Rejects an empty checkout, invalid quantity, and an overlong address before database access.	Successful MySQL order persistence, transaction rollback after a mid-write failure, stock contention, or API auth.
Java `CartTest` (1 test)	Cart subtotal and total quantity calculations.	Browser cart behavior or API/database cart integration.
Java `RoleTest` (1 test)	Role enum spelling/values expected by the schema.	Authorization coverage for every route.
Nested frontend `cart.test.ts`	Subtotal, estimated delivery/total, empty cart, and quantity clamp.	API/auth/checkout/admin integration.
Nested frontend `app-routing.test.tsx`	Minimal route-match smoke test.	Full page behavior, API calls, or browser compatibility.

The nested frontend test suite is small. The root frontend has no test script. No SQL integration test suite is configured in the inspected source.

Recorded verification evidence

The latest project checks recorded before this documentation task reported:

- Nested frontend TypeScript check, tests (2 files / 5 tests), and production build passed.
- Java sources and tests compiled directly with Java 17 and cached dependencies; 5/5 Java unit tests passed. Maven was not available in that environment, so no Maven pass was claimed.
- Live Java API/MySQL smoke checks succeeded for health, public restaurants (7 records), a menu read, a temporary customer registration/login, incorrect-password rejection, customer order-history access, customer denial at an admin route, invalid-item checkout rejection, logout, and token revocation. The temporary customer was removed.
- That smoke test did **not** create a valid order or perform admin writes. Earlier recorded checkout work verified a valid order persisted across an API restart; this documentation task did not repeat that test.

These are historical results from project work, not tests run while generating this guide. The SQL routines and views were inspected but not executed during this documentation task.

Troubleshooting checklist

Symptom	Check
API health cannot connect	Start MySQL first, then the Java API. Confirm port `8080` (or configured `api.port`) and check the API console for a sanitized startup error.
Frontend gets 404/connection refused	Start the Java API, verify `/api/health`, then confirm the selected frontend Vite proxy still targets the same API port.
`java -jar` opens desktop window	Expected from the manifest. Start the API with `java -cp ... com.foodapp.api.ApiMain`.
DB access denied	Confirm the private root `config.properties` or DB environment variables match the MySQL account; do not print values while debugging. Java property values override environment fallbacks when nonblank.
Registration/login rejected	Check the request field names, user active status, backend role, and BCrypt path. Never "fix" login by storing plaintext passwords or bypassing checks.
Admin page loads but write returns 401/403	Check that the current session is valid and its server-side role is ADMIN; a client-side role toggle is not authorization.
Checkout rejects item/stock	Refetch current menu/availability and use valid positive quantities. Server prices and inventory are authoritative.
SQL seed reports duplicate index	`indexes.sql` is not idempotent for already-created indexes; inspect the actual schema rather than rerunning blindly.
Mock appears inconsistent	Turn off `VITE_USE_MOCK`; mock checkout and some admin handlers do not fully match the API.

Known gaps and limitations

- Two separately maintained React apps can confuse users; choose `frontend/foodos-frontend/` for the intended customer/admin API client.
- Both browser carts are local browser state even though backend cart routes are available.
- Nested mock checkout/admin behavior has contract gaps and should not be used as evidence of live API support.
- API sessions are stored in process memory and expire after eight hours; a server restart invalidates them.
- CORS is broad in the current Java handler; production origins should be restricted.
- Payment processing is simulated/COD, not a live gateway.
- Several SQL views/functions/procedures are not called by the Java application according to source search.
- Existing docs contain older claims about persisted web carts, status of verification, or mandatory one-to-one inventory/payment; use the source and schema in this guide as authority.

16. Security and data-handling basics

Passwords

The service uses BCrypt hashing/verification. A secure system stores a salted password hash, never the user's original password. Hashes and passwords must not be sent in API responses, logs, screenshots, or documentation. Seed scripts contain sample hashes; this guide deliberately omits them.

Authentication and authorization

Login issues a bearer token held by `SessionStore` for eight hours. The browser sends it in an Authorization header for protected calls. Logout revokes it. Because the store is in process memory, server restart clears sessions. Role checks happen at the API boundary. Do not depend on route guards or hidden buttons for security.

SQL and input validation

- Use `PreparedStatement` for values.
- Validate body shape, required fields, allowed enums, quantity, lengths, and IDs before writes.
- Derive customer identity from the authenticated session.
- Validate that an address belongs to the requesting customer.
- Re-read menu availability, restaurant status, inventory, and price at checkout.
- Keep checkout writes in one transaction and roll back on every failure.
- Return safe client errors; log operational details only in appropriately protected backend logs.

Secrets and deployment

- `config.properties` is local runtime configuration. Keep real values out of Git.
- Do not place DB credentials in `VITE_*` variables; Vite variables are exposed to browser code.
- Use environment/secret management in production, a least-privilege DB account, TLS for deployed traffic, restricted CORS, and backups.
- Do not commit real `.env` files or bearer tokens. Keep `.env.example` limited to safe defaults/placeholders.
- Do not point development seed scripts at production data.

17. Glossary, revision notes, and viva questions

Glossary

Term	Beginner definition
API	A documented way for one program (React) to ask another (Java) to do work.
Backend	Server-side code that validates requests and accesses trusted data.
CORS	Browser rule controlling which web origins may call a server.
DAO	Data Access Object; class/interface responsible for database operations.
Foreign key	Database rule requiring a reference to an existing row in another table.
HTTP	Request/response protocol used by the browser and Java server.
Idempotent	Safe to repeat without unintended additional effects; not every SQL script here is idempotent.
InnoDB	MySQL storage engine supporting transactions and foreign keys.
JSON	Text format for structured API request and response values.
JDBC	Java's standard API for connecting to relational databases.
Primary key	Column(s) uniquely identifying a row.
Prepared statement	Parameterized SQL statement with values bound separately.
REST	Resource-oriented API design that commonly uses HTTP methods such as GET, POST, PUT, DELETE.

Term	Beginner definition
ResultSet	JDBC representation of rows returned by a SELECT query.
Transaction	Group of database operations that commit together or roll back together.
TypeScript	JavaScript with static type checking.
Vite proxy	Local dev-server forwarding rule; it does not itself deploy the backend.

Revision notes

- React performs presentation and user interaction; Java is responsible for trusted validation and DB access.
- The intended frontend is `frontend/foodos-frontend/`; `src/` is another separate frontend.
- Registration and login are different requests. Registration returns a message; login returns user fields and a token.
- A customer browser cart is not the same as the backend SQL cart.
- The database price, restaurant status, availability, and inventory must be rechecked at checkout.
- Order line prices are historical snapshots; changing a menu price later should not rewrite past orders.
- SQL constraints and backend validation complement one another; neither substitutes for the other.
- `mvn test` and frontend tests cover only the tests actually present. Do not infer full-system certification.

Project viva / interview questions with answers

1. What are the main layers in FoodOS? The React UI, Java API/services/DAOs, and MySQL database. There is also a separate Swing desktop client.

2. Why does the browser call Java instead of MySQL directly? The server keeps database credentials private, enforces business rules, and applies authorization.

3. What is the purpose of `ApiHandler`? It maps HTTP method/path/body to application operations, checks access, and maps results/errors back into HTTP responses.

4. Why does `OrderService` use a transaction? Inventory decrement, order, order lines, and payment must be all saved or all rolled back. A partial order would be inconsistent.

5. Why re-read price at checkout? The browser's displayed amount can be stale or manipulated. The server must calculate totals from current database values.

6. What prevents SQL injection in a DAO? Parameterized `PreparedStatement` values prevent user text from being interpreted as SQL syntax.

7. What is a foreign key? Give an example. `menu_items.category_id` refers to `menu_categories.category_id`, ensuring each menu item belongs to a real category.

8. Why are both `UserDao` and `JdbcUserDao` present? The interface defines the contract; the JDBC class implements it with MySQL queries. Services can depend on the abstraction.

9. Does the UI's admin route check secure the API? No. The Java API must check the token and admin role for every protected request, because clients can be bypassed.

10. Is the web cart stored in the `carts` table? Not in the inspected React flows. The browsers keep cart state locally and send item IDs/quantities at checkout. Separate backend cart endpoints exist.

11. What does HTTP 401 mean compared with 403? 401 means the request lacks a valid authenticated session; 403 means the session is valid but its role is not allowed.

12. Why does `order_items` store `unit_price`? It records the price charged at the time of purchase, so a later menu-price edit does not change historical order totals.

13. What is `line_total`? A generated stored SQL column equal to `quantity * unit_price`.

14. What happens to API sessions after a server restart? The process-local session map is lost, so users must log in again.

15. Is every SQL file safe to run more than once? No. For example, `indexes.sql` uses plain `CREATE INDEX`, and catalog seed statements can alter inventory. Inspect scripts and database state first.

16. Which frontend is intended for the customer/admin API? `frontend/foodos-frontend/`, the React 19/TanStack app. The project-root React 18 app is separate.

17. What do the current unit tests prove? They cover selected checkout input validation, cart arithmetic, role names, a nested cart helper, and minimal route matching. They do not prove every API/database/UI workflow.

18. How should a developer run the web API rather than the desktop app? Run the Maven-built classpath command with `com.foodapp.api.ApiMain`; the shaded JAR's manifest defaults to `com.foodapp.Main`.

Sources and change discipline

Primary evidence: `pom.xml`, both frontend manifests and Vite configs, Java production/test source, `database/schema.sql`, the other SQL scripts, and existing project documentation. Read the source again when the project changes. If source and prose disagree, trust the active code/schema and update this guide.

No application source, SQL data, dependency manifest, or database record was changed while authoring the guide. The deliverables are this Markdown file and its rendered PDF.